Python MC Modpack Server Local Hoster: Plan of Action (v3)

A Python tool with a localhost web dashboard for hosting Minecraft modpack servers that players can join from anywhere.

| | = V1

| | = V2

| | = V3

Overview

One small Python program runs on the host computer. It opens a dashboard at something like `http://localhost:8080`, and everything else (creating, configuring, and managing Minecraft servers) happens in the browser. The program launches and supervises the actual Minecraft server processes in the background, and players on any internet-connected device can join them.

Decisions so far

- **Loaders:** Fabric, NeoForge, Quilt, and Paper.
- **Pack source:** Modrinth.
- **Interface:** a localhost web dashboard.
- **Internet access:** players on any device should be able to connect.
- **Connection method:** Dynamic DNS with a custom domain.
- **Privacy goal:** the host does not want their home IP exposed. See the Internet access section, because a domain alone does not achieve this.

Features list

Server creation and setup

- Create servers for Fabric, Quilt, NeoForge, and Paper from one wizard.
- Search and install Modrinth modpacks, or import a local `.mrpack` file.

- Automatic Java detection, with a warning or download when the wrong version is installed.
- Hash-verified mod downloads, with client-only mods skipped automatically.
- EULA acceptance step and a friendly editor for `server.properties`.
- RAM and port settings per instance, with port conflict detection.

Running and monitoring

- *Start, stop, and restart servers from the dashboard.*
- Live console with command input and command history.
- Status cards showing state, player count, and resource use.
- Crash detection with optional auto-restart and crash report viewing.
- Multiple servers running at the same time.

Content management

- Add, remove, and update mods or plugins by Modrinth slug.
- Update checker for installed mods and plugins.
- Whitelist, ops, and ban list management.
- Optional file browser for configs and worlds.

Backups

- One-click world backups, plus an optional schedule.
- Restore and delete backups from the dashboard.

Connectivity and sharing

- Players can join from any internet-connected device.
- *Shows the local address for LAN* players and the public address for remote players.
- Dynamic DNS updater that keeps a custom domain pointed at the host.
- Reachability check that tells you whether the server can actually be joined from outside.
- Optional tunnel or relay mode for hiding the home IP.
- Optional Bedrock crossplay through Geyser.

Security and quality of life

- Dashboard private by default, with a login required if it is ever exposed.
- *Graceful shutdown of all servers when the program quits.*
- Clear error messages for common failures (wrong Java, port in use, failed download).
- Optional system tray icon or background service mode.

How it works

1. Run the program on the computer that will host the servers.
2. It starts a local web server (the dashboard) and opens your browser to it.
3. You create and manage Minecraft servers from that page, while the program runs them in the background.

There are two separate things on two separate ports: the dashboard (for example 8080) and each Minecraft server (25565 by default, then 25566 and so on). They must never be confused in the UI, and only the Minecraft ports are ever opened to the internet.

Tech choices

- **Backend:** *FastAPI. It handles async work and WebSockets (needed for the live console) and generates an API for free.* Flask also works but WebSockets are clunkier.
- **Frontend:** *plain HTML plus a light library like htmx or Alpine.js, so no Node build step is needed.* Move to React or Vue only if the UI gets complicated.
- **Live updates:** WebSockets for the console and install progress. Server-sent events are a simpler alternative for one-way streaming.
- **Storage:** *one JSON or SQLite file for instance settings, with each server's files in its own folder*.

Architecture

- *API layer: the web pages talk to a JSON API (for example /api/instances and /api/instances/{id}/start). Because the UI is only a client of that API, a CLI or different front end could be added later without rewrites.*
- **Core layer:** loader installers, the Modrinth client, Java detection, and process management. Nothing in it should know the web exists.
- **Background tasks:** installs and downloads run as background jobs with progress reported over WebSocket. One supervisor object tracks every running server process.
- **Suggested layout:** `cli.py` (entry point), `config.py`, `instances.py`, `java.py`, `loaders/` (fabric, quilt, neoforge, paper), `packs/modrinth.py`, `server.py`, `properties.py`, `ddns.py`, `utils.py`, plus an `instances/<name>/` folder per server.

Loaders

Fabric, Quilt, and NeoForge run **mods**. Paper runs **plugins** (Bukkit and Spigot API) and cannot load Fabric or NeoForge mods, so treat Paper as a separate instance type. Quilt can load most Fabric mods, so the Quilt path can fall back to Fabric-tagged files when a mod has no Quilt build.

Give every loader the same small interface (list versions, install, build the launch command) so the rest of the program does not care which one it is.

- *Fabric: the Fabric meta API serves a ready-made server launch jar for a given Minecraft, loader, and installer version, so there is no installer step.*
- **Quilt:** download the Quilt installer and run it in server mode with the Minecraft version. It produces a launch jar.
- **NeoForge:** download the installer from NeoForge's Maven and run it with the install-server option. Modern versions launch through generated arguments files rather than a single jar, so the launch command must handle the Windows and Linux variants.
- **Paper:** download the build jar from PaperMC's downloads API and run it directly. Check their current API documentation, since endpoints have changed between versions.

Modrinth integration

1. **Resolve the pack:** query the project's versions, filtered by loader and game version, and download the latest release's `.mrpack` file.
2. **Read the index:** `modrinth.index.json` lists every file with download URLs and hashes, the dependencies (Minecraft version, loader, loader version), and a per-file environment field.
3. **Filter:** skip files whose server environment is marked unsupported.
4. **Download and verify:** check each hash, and download in parallel with a thread pool while respecting rate limits.
5. **Overrides:** extract the `overrides` folder and then `server-overrides` on top of the instance folder.
6. **User-Agent:** send a descriptive User-Agent header on every API call, as Modrinth asks.

For Paper instances, skip the modpack flow and let users add plugins by slug, filtered by the Paper loader and the instance's Minecraft version, checking each plugin's dependencies.

Dashboard pages

1. **Dashboard:** all instances as cards showing status, loader, Minecraft version, player count, and Start and Stop buttons.
2. *Create server: choose type, search Modrinth packs, pick versions, set RAM and port, accept the EULA*, and watch a live install log.
3. **Instance page:** tabs for Console, Mods or Plugins, Settings (a friendly `server.properties` editor), Backups, and later an optional Files browser.
4. **App settings:** Java location, default RAM, instance storage folder, dashboard port, and Dynamic DNS configuration.

Internet access and your domain

There are two different things to open up and they should be treated differently: the Minecraft servers (so players can join) and the dashboard (so you can manage things). The plan opens only the Minecraft ports and keeps the dashboard private unless you choose otherwise.

Chosen path: Dynamic DNS with a custom domain

1. Buy a domain from a registrar and manage its DNS somewhere with an API, such as Cloudflare's free DNS.
2. Create an A record for something like `play.yourdomain.com`.
3. The program checks your public IP every few minutes and updates that record through the DNS provider's API whenever it changes. This is a built-in Dynamic DNS settings page where you paste an API token and pick the record.
4. Add an SRV record so players can leave off non-default ports.
5. Forward the Minecraft port on your router to the host computer and allow it through the operating system firewall.
6. Set the DNS record to "DNS only", not proxied. Cloudflare's normal proxy only handles web traffic and would break Minecraft connections.
7. Never put the dashboard on this domain.

Important: a domain does not hide your IP

A domain is only a name that points at an address. With Dynamic DNS, `play.yourdomain.com` points at your home IP, and anyone can look that up in seconds. When a player connects, their computer also talks directly to your IP. A domain gives players a friendly name and keeps the address working when your IP changes, but it does not hide anything.

If hiding your home IP matters

You need something in the middle that holds the public IP instead of your home connection.

- **Tunnel service (playit.gg and similar):** players connect to the provider's address and traffic is relayed to your computer. You can often point your own domain at it with a CNAME. No router changes are needed, and it works with CGNAT. The trade-off is relying on a third party and possible free-tier limits.
- **Cheap VPS as a relay:** rent a small cloud server, point your domain at its IP, and forward Minecraft traffic to your home computer through a private link such as WireGuard or frp. Your home IP is never published. This is the most robust option but costs a few dollars a month and takes more setup.

Both replace the plain Dynamic DNS approach, since the home IP no longer needs tracking. The program can offer them as an alternative mode.

Check for CGNAT first

If your ISP uses CGNAT (shared public IPs), port forwarding and Dynamic DNS will not work at all, and a tunnel or relay becomes the only option. To check, compare the WAN IP shown in your router with what a "what is my IP" site reports. If they differ, you are behind CGNAT. The program should detect this and recommend the tunnel option.

What the program should do

- Detect and show the local IP, the public IP, and the domain address next to each server.
- Run a port check to confirm the server is reachable from outside.
- Walk through router port forwarding, since every router differs. UPnP auto-forwarding is possible but should be opt-in and clearly disclosed.
- Give each instance its own port and show the exact address players should type.
- Support Bedrock players (phones and consoles) as an optional add-on through Geyser, which needs UDP port 19132 open.

Security

A web dashboard that can start processes and edit files is a real attack surface, even on localhost, and opening Minecraft ports adds more.

- *Bind the dashboard to 127.0.0.1 only by default, not 0.0.0.0.*
- **Remote dashboard access** is an explicit opt-in setting that requires a login and HTTPS. A private network like Tailscale is safer than exposing the dashboard to the open internet.
- **Protect against malicious web pages** poking at the localhost API by requiring a token or checking the Origin header on state-changing requests.
- **Sanitize file paths** in any file browser or backup feature so nothing can escape the instance folder.
- **Console commands** go to the Minecraft server only, never to a system shell.
- **Turn the whitelist on by default** and keep online-mode enabled, so only real authenticated accounts can join.
- **Expect scanning and DDoS attempts** once an IP is public, and say so in the UI.
- **Keep server software and mods updated**, because some mods have had remote exploits.
- **Never forward the dashboard port by accident.** Only Minecraft ports should be opened on the router.

Closing the tab versus closing the program

Closing the browser tab should not stop your servers, because the Python program owns them. Closing the program would, so the behavior is:

- Running the program keeps servers alive, and the browser is optional.

- When quitting, ask or default to gracefully stopping all servers (send `stop`, wait, then force-kill after a timeout).
- Later, optionally add a system tray icon or background service so servers survive without a visible window.

Build order

1. Skeleton FastAPI app serving a basic page on localhost, with a settings file.
2. Core: Java detection, Fabric install, and start and stop of one server through the API.
3. Live console over WebSocket.
4. Dashboard and create-server page with Modrinth pack import and progress streaming.
5. Settings editor and Mods or Plugins tab.
6. Dynamic DNS updater, public IP detection, and the reachability check.
7. Backups.
8. Quilt, NeoForge, and Paper support.
9. Optional tunnel or relay mode and Bedrock crossplay.
10. Security pass (token, origin checks, path safety), then packaging so it runs with one command or one executable.

Libraries worth using

`fastapi` and `uvicorn` (server), `httpx` (downloads and DNS API calls), `pydantic` (config models), `psutil` (process and resource stats), `rich` (terminal output), and `watchdog` (optional log tailing).